



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 1 · Введение. Изображение и видео как данные

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиплярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L1: Введение. Изображение и видео как данные

RECAP

- Курс вырос из нескольких лет лекций по компьютерному зрению и робототехнике
- Зрение — это **измерительная** дисциплина: ответ в метрах, градусах и пикселях, а не «похоже / не похоже». Поэтому начинаем не с «распознай котика», а с измерений.
- Вопрос курса: как из массива чисел (пикселей) извлечь **геометрию сцены**
- Нейросетей не будет: они начинаются там, где этот курс заканчивается, и стоят на нашем классическом фундаменте
- Каждое утверждение курса проверяется **измерением** — включая распределение памяти под изображения, преобразования пикселей, и т.п.

LEARNING OBJECTIVES

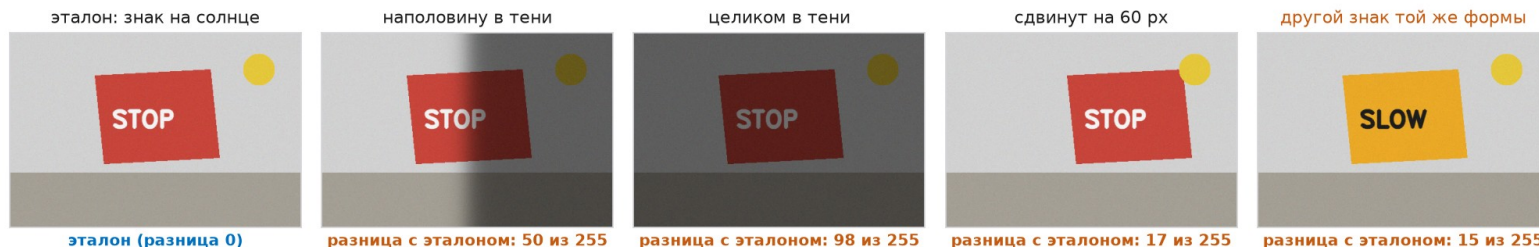
- **Объяснять**, почему изображения не стоит сравнивать попиксельно и что такое обратная задача
- **Называть** пять характеристик массива изображения и **связывать** каждую с физикой съёмки
- **Прочитывать** код изображения и видео и **измерять** их параметры, не веря заголовку файла
- **Находить** классические ошибки новичка в чужом фрагменте кода
- **Выбирать** флаг `resize` и формат файла по измерению, а не по привычке

МОТИВАЦИЯ

Два кадра одного дорожного знака. Как понять из матрицы: один ли это знак?

- Человеку понять это — доля секунды, без вычислений и ошибок
- Попиксельная разница: тень даёт **98** из 255, сдвиг — **17**. То есть освещение меняет пиксели изображения сильнее, чем подмена объекта — другого дорожного знака (всего **15**)!
- Робототехника: проблемы для мобильных роботов и автономного транспорта — где есть препятствия на дороге в тени/на солнце и др. критичные объекты
- МОИИ: задача об **инвариантах** — что не меняется, когда меняется всё

Для человека четыре кадра слева — один знак. Для массива тень (98) меняет больше, чем сдвиг (17) и даже чем другой знак (15)



© собственная иллюстрация

1

Зрение: человеческое и машинное

- Что получает алгоритм, чего у него нет, и почему это обратная задача

Человек видит, алгоритм получает числа

что видит человек: здание, окна, небо



фрагмент 10 × 10 пикселей



что получает алгоритм: 100 чисел 0...255

210	213	204	189	40	26	15	13	23	40
169	174	188	202	57	36	25	20	16	30
216	216	219	206	97	38	44	34	22	22
211	207	206	199	149	26	23	24	28	28
209	205	204	201	182	23	14	6	9	37
209	205	211	203	200	48	15	6	12	106
203	203	210	205	205	84	16	10	23	178
202	206	210	199	199	115	15	13	39	199
206	208	216	201	198	152	8	16	65	203
205	209	209	203	206	141	10	23	89	215

Алгоритму не даны ни «окна», ни «здание» — только 2.5 миллиона чисел от 0 до 255. Всё остальное придётся вычислить. Причём вычислить в ситуации, когда одна и та же сцена даёт совершенно разные массивы при другом свете, ракурсе, дистанции. Именно поэтому зрение — обратная задача: нужно восстановить объект из матрицы

© фото — здание Университета Иннополис; собственная иллюстрация

Наблюдать, показывать, измерять

Прямое наблюдение



Фото и видео для человека



Компьютерное зрение

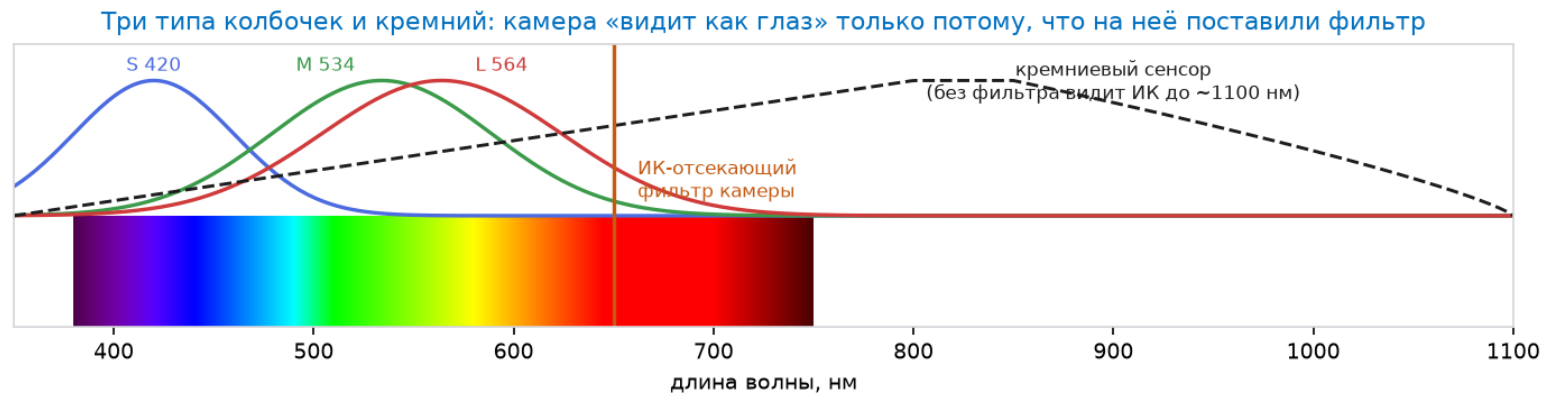


В третьей цепочке потребитель кадра — не человек, а алгоритм. Поэтому «хорошая картинка» и «хорошие данные» — разные вещи

В цепочке компьютерного зрения потребитель кадра — не человек, а алгоритм. Поэтому «красивая картинка» и «данные, пригодные для измерения» — разные критерии: кадр, приятный глазу после агрессивного шарпенинга, для измерений может быть бесполезен. В курсе мы смотрим на кадр глазами алгоритма: как на результат измерения. И первый вопрос измерения — что именно измеряет пиксель? Спойлер: не «цвет».

© собственная иллюстрация

Свет: что именно измеряет пиксель

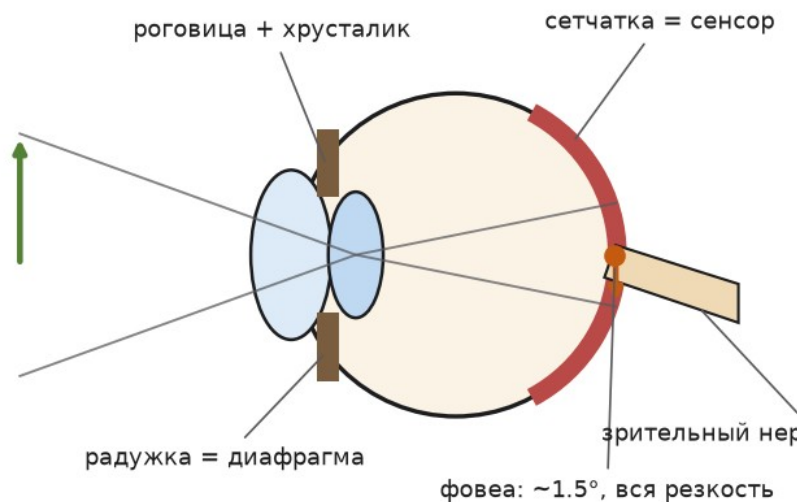


Пиксель — не «цвет», а интеграл спектра $I = \int L(\lambda) \cdot S(\lambda) d\lambda$, взвешенный чувствительностью сенсора. Три интеграла (R, G, B) спектр не восстанавливают — разные спектры дают те же числа (метамерия, metamerism). Кремний видит ИК до 1100 нм: «как глаз» камера видит только благодаря фильтру. Подробно — в лекциях L2 и L3

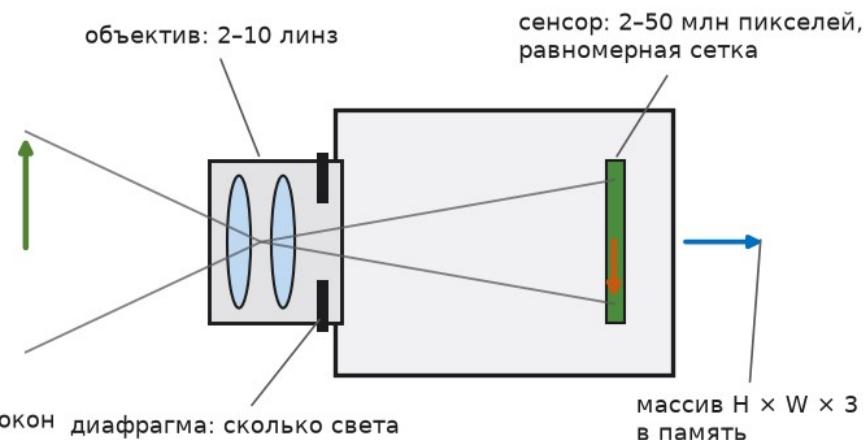
© собственная иллюстрация

Глаз и камера: где кончается аналогия

Глаз: ~120 млн палочек + 6 млн колбочек,
неравномерная сетка и мозг за ней



Камера: та же оптика, но однородный сенсор
и никакого «мозга» после него



Привычная аналогия «глаз и камера». Где она ломается? Аналогия работает до сенсора (линза, диафрагма, светочувствительная поверхность) и ломается после: у камеры нет ни фовеа (fovea — зоны «наилучшего видения»), ни саккад (saccade — механизм наведения на фовеа), ни зрительной коры, которая дорисовывает мир по десятку фиксаций. Глаз — не камера, а сканер с мощной постобработкой. Роль коры в ваших программах играет ваш код.

© собственная иллюстрация

Одна фиксация: резко только 1.5°

как нам кажется: резко везде



что даёт одна фиксация: резко только $\sim 1.5^\circ$ вокруг точки взгляда



Глаз делает 3–4 скачка (саккады) в секунду и собирает картину из фрагментов. Камера снимает всё поле сразу и одинаково — за это платит шумом и объёмом данных

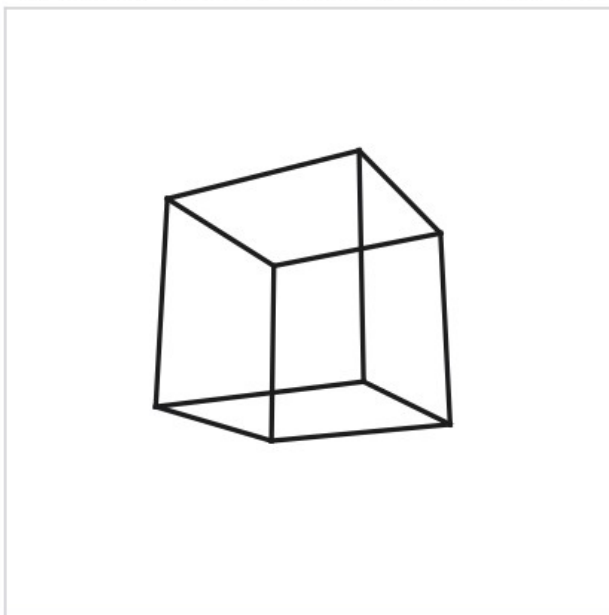
За одну фиксацию глаз видит резко лишь 1.5° — остальное периферия, которую мозг «дорисовывает». Т.о., глаз — это сканер с одной резкой точкой и мощной постобработкой; камера — равномерная сетка без постобработки. Отсюда разные недостатки: глаз обманывает иллюзия, камеру — тень, ей нечем дорисовать. Вывод: не опирайтесь на аналогию сильно. Камера — не глаз, а измерительный прибор. И как всякий прибор, она ставит обратную задачу.

© фото — здание Университета Иннополис; собственная иллюстрация

Обратная задача: снимок один, сцен много

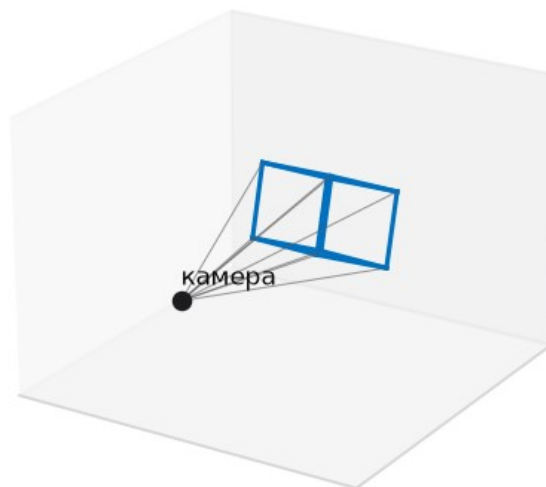
Обе сцены дают в точности этот снимок: по одному изображению глубина не восстанавливается

снимок: 12 отрезков на плоскости



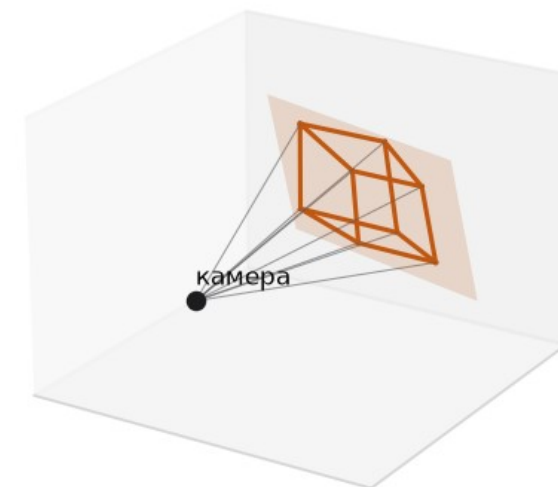
сцена А:

куб на расстоянии от камеры



сцена Б:

плоский рисунок на наклонном листе



Формирование изображения — прямая задача физики: сцена $\rightarrow$ однозначный снимок. Зрение — обратная задача: по снимку восстановить сцену (снимок $\rightarrow$ сцена), и решений бесконечно много. Нужен второй вид (блок лекций IV) или допущения о мире: гладкость, жёсткость, плоскость (весь остальной курс)

© собственная иллюстрация

Не «распознать картинку», а измерить

- **Где** объект: координаты, поза, расстояние — в пикселях и в метрах
- **Какой** он: размер, форма, ориентация, состояние
- **Как движется**: скорость, направление, прогноз на следующий кадр
- **Как устроена сцена**: что ближе, что заслоняет, где плоскость пола
- **Что изображено** на сцене: какие объекты на сцене (классификация) и понимание сцены

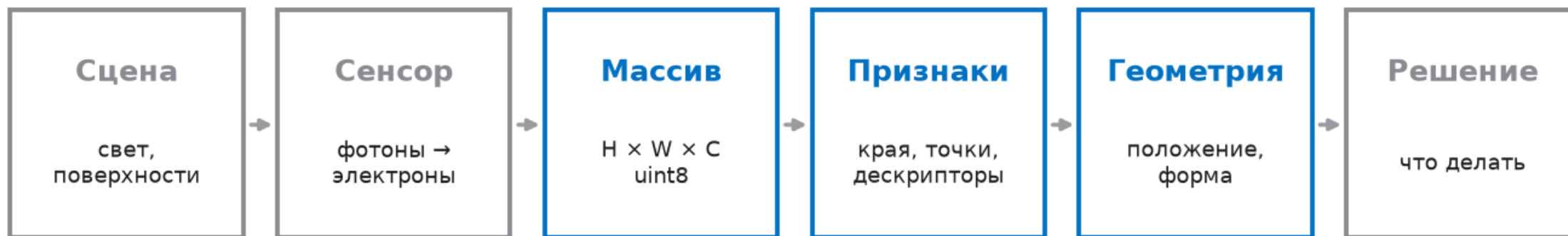
Классификация — что это за объекты на сцене и понимание сцены — единственные задачи, где нейросеть вне конкуренции

Источники вариативности

- Освещение меняет пиксели **сильнее**, чем смена объекта: тень на 98, сдвиг на 17 из 255
- Ракурс, масштаб, поворот меняют **положение** каждого пикселя, а не его значение
- Заслонение (occlusion) и фон: части объекта нет, вместо неё — чужие пиксели
- Итог: один объект — миллионы разных массивов. Значит, нужны **признаки**, устойчивые (инвариантные) к этому списку: про это блок III курса

Пять источников вариативности: свет, ракурс, масштаб, заслонение, фон

Но вариативность — не единственная беда: информация теряется ещё в тракте съёмки



Курс живёт вот здесь: от массива чисел до геометрии сцены

Пайплайн: где теряется информация. Цепочка: сцена → оптика → сенсор → кодек → алгоритм. Главное: каждое звено теряет информацию, и назад её не вернуть. Пересвеченный сенсором кадр не спасёт никакая обработка; смаз — тоже. Курс занимает три средних звена: сенсор и оптика — блок I, операции — блок II, алгоритмы — блоки III–V. Запомните иерархию потерь: чем раньше информация потеряна, тем фатальнее.

© собственная иллюстрация

Где классика работает в 2026 году

Где	Что делает классика	Лекции
Панорама в телефоне	признаки → гомография → сшивка	L9–L10
AR-очки, ARKit / ARCore	калибровка, PnP, визуально-инерциальная одометрия	L11, L15
Марсоходы NASA	стерео и визуальная одометрия по углам Харриса	L8, L12, L15
Контроль качества на конвейере	сопоставление с эталоном, морфология, измерение	L3, L5
Спортивная аналитика	гомография поля → метры, трекинг, фильтр Калмана	L10, L14

Всё это прямо сейчас работает в продакшне, на борту, без GPU. А где граница с нейросетями?

Нейросеть даёт

- **Семантику:** «это пешеход»
- Соответствия там, где текстура бедна
- То, что трудно описать формулой
- Восстановление (генерация текстур) изображения (диффузионные модели)

Классика даёт

- Числа **в метрах:** калибровка, PnP, триангуляция
- Объяснимость: видно, где и почему сломалось
- Работу без датасета и без GPU

Пайплайн 2026 — гибрид: сеть находит, геометрия измеряет

Этот курс — фундамент, на котором гибрид стоит

Полвека зрения на одном слайде



Инструментарий курса сложился в 1981–2011: Lucas–Kanade, RANSAC, Canny, Harris, Zhang calibration, SIFT, SGM, ORB. После 2012 нейросети заменили распознавание, но не геометрию: эпиполярная геометрия и гомографии никуда не делись. То, с чем мы познакомимся на курсе, имеет срок годности десятилетия, а не сезоны.

© собственная иллюстрация

- **NumPy** — контейнер: изображение это `ndarray`, и ничего больше; на свойствах массива построена вся логика курса
- **OpenCV** (Open Source Computer Vision Library) — самая популярная в мире открытая библиотека ПО для компьютерного зрения и машинного обучения. Проект зародился в 1999 году по инициативе Intel. 1-й публичный релиз (бета-версия) — в 2000 году на CVPR. Полноценная версия OpenCV 1.0 — конец 2006. Идея создания — Гари Брадски (Intel), разработка — команда российских инженеров и математиков в Нижнем Новгороде (выпускники ННГУ им. Лобачевского). В середине 2000-х годов Intel заморозил OpenCV. Чтобы проект не погиб, нижегородские выходцы из Intel основали в Нижнем Новгороде компанию Itseez, превратив OpenCV в мировой стандарт. В 2016 году Intel выкупил Itseez.
- **OpenCV** — 2500+ алгоритмов на C++ (Intel, 1999), вызываемых из Python **без копирования** данных
- Одна функция — миллисекунды; цикл по пикселям на Python — **сотни** миллисекунд
- Отсюда и удобство, и ловушки: срез меняет оригинал, `uint8` переполняется молча

- **Магии:** каждая функция — алгоритм с параметрами, и значения по умолчанию подобраны не под вашу задачу
- **Проверок:** `cv2.imread` вернёт `None` вместо ошибки, `uint8` переполнится молча
- **Стабильности между ветками:** в 5.0 `HoughLinesP` возвращает массив другой формы

Курс зафиксирован на **OpenCV 4.14.0.94**. Окружение проверяется командой: `python tools/check_env.py`

Робототехника

- Калибровка и поза камеры — L11
- Стерео и глубина в метрах — L12
- Поток, трекинг, одометрия — L13–L15
- Всё это работает **на борту**, без облака и GPU

МОИИ

- Признаки **до того**, как их стали учить — блок лекций III
- Геометрия как инвариант и как индуктивное допущение
- Аугментации — это блоки I и II
- Честная валидация: метрика с размерностью

- Зрение — **обратная задача**: снимок теряет глубину, спектр и время; вернуть их — цель курса
- Алгоритм получает только числа; всё, что «видно» человеку, придётся вычислить, и попиксельно это не делается
- Классика измеряет, нейросети распознают; в 2026 году они работают вместе (гибрид)

Дальше — четыре демонстрации, и с них начинается вторая часть: изображение как данные

Демо 1.1 — первое изображение

```
1 import cv2
2 from cvcourse import io as cio, viz
3
4 img = cio.imread("demo/iu-building.jpg") # массив, не «картинка»
5 print("shape ", img.shape)             # (строки H, столбцы W, каналы C)
6 print("dtype ", img.dtype)             # тип одного значения
7 print("min/max", img.min(), img.max())
8 print("bytes ", img.nbytes)            # H · W · C · 1 байт
9 viz.show(img, "как OpenCV увидел файл")
```

python

shape (1279, 1920, 3) · dtype uint8 · min/max 0
255 · bytes 7 367 040
1279 × 1920 × 3 × 1 байт = 7 367 040 байт
Когда функция cv2.imread() считает массив
для работы алгоритмов OpenCV, он
развернётся в памяти в массив ≈ 7.3 МБ

Вызов	Что делает
<code>cio.imread(path)</code>	читает файл в массив (порядок каналов B, G, R); при ошибке — исключение, а не None
<code>img.shape, img.dtype, img.nbytes</code>	форма (H, W, C), тип одного значения, размер в памяти — свойства массива, не OpenCV
<code>viz.show(img)</code>	показывает массив, переставляя каналы в RGB; работает и в Colab

Ошибка с неверным путём к файлу (None вместо ошибки) — 1-я классическая ошибка для новичков в OpenCV. Функция cv2.imread() не выдаст ошибку, а вернёт None. Код упадёт только на следующей строчке (e.g., img.shape)

Демо 1.2 — ловушка BGR в числах

python

```
1 y, x = 897, 857          # строка y, потом столбец x
2 b, g, r = img[y, x]      # пиксель оранжевого фасада
3 print("канал 0 =", b, " канал 1 =", g, " канал 2 =", r)
4
5 rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # BGR → RGB
6 print("тот же пиксель после cvtColor:", rgb[y, x])
```

канал 0 = 81; канал 1 = 130; канал 2 = 206;
после cvtColor: [206 130 81]

Вызов	Что делает
<code>img[y, x]</code>	три числа одного пикселя; индекс — строка, затем столбец, то есть (y, x), а не (x, y)
<code>cv2.cvtColor(img, код)</code>	пересчитывает цветовое пространство; COLOR_BGR2RGB просто меняет порядок каналов

Обратный порядок BGR (вместо RGB) — вторая классическая ошибка для новых пользователей OpenCV

Демо 1.3 — uint8 переполняется молча

```
1 import numpy as np
2 a = np.uint8([[200]])          # «картинка» 1 × 1
3 print((a + np.uint8(100)).ravel()) # NumPy: по модулю 256
4 print((a - np.uint8(210)).ravel()) # минус стал плюсом
5 print(cv2.add(a, np.uint8([[100]])).ravel()) # OpenCV: насыщение
6
7 bright = img.astype(np.float32) * 1.5      # считаем во float32...
8 bright = np.clip(bright, 0, 255).astype(np.uint8) # ...и обрезаем
```

python

NumPy: $200 + 100 = [44]$; $200 - 210 = [246]$;
OpenCV `cv2.add` = `[255]` · bright: uint8, max 255
uint8: $200 + 100 = 44$ — арифметика по модулю 256, без предупреждения.
 $200 - 210 = 246$ — минус стал плюсом.
То же сложение через `cv2.add` даёт 255: OpenCV насыщает.

Вызов	Что делает
<code>a + b</code> в NumPy	арифметика по модулю 256 для uint8: переполнение без предупреждения
<code>cv2.add(a, b)</code>	то же сложение, но с насыщением : всё, что больше 255, становится 255
<code>astype(np.float32) → np.clip → astype(np.uint8)</code>	правило курса: считать во float, обрезать, вернуть в байт

Правило данного курса — любая арифметика над изображением идёт через float, потом обрезка (clip), потом возвращение в байт

Демо 1.4 — цикл против NumPy

```
1 import time
2 g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY) # один канал
3 t0 = time.perf_counter()
4 s = 0
5 for y in range(g.shape[0]): # 2.45 млн итераций на Python
6     for x in range(g.shape[1]):
7         s += int(g[y, x])
8 t1 = time.perf_counter()
9 m = g.mean() # то же самое в NumPy
10 t2 = time.perf_counter()
11 print("цикл %.0f мс" % ((t1 - t0) * 1e3))
12 print("NumPy %.2f мс" % ((t2 - t1) * 1e3))
```

python

цикл: 262 мс; NumPy: 1.67 мс; ответ одинаков: True;
разница ≈ 150 раз
На примере вычисления среднего значения яркости:
Цикл Python: выполняется за 262 мс.
Векторизованный NumPy (`g.mean()`): выполняется за 1.67 мс.
Результат: NumPy 150× быстрее (компиляция на C + CPU-оптимизация).

Вызов

`cv2.cvtColor(..., COLOR_BGR2GRAY)`

`time.perf_counter()`

`g.mean()`

Что делает

сворачивает три канала в один по формуле яркости $0.299 R + 0.587 G + 0.114 B$

точные часы для замеров; разность двух вызовов — время в секундах

среднее по всем элементам одним вызовом — цикл выполняется в C, а не в Python

Никогда не используйте стандартные циклы Python (`for`) для построчной и попиксельной обработки изображений. Проводить математические операции над матрицами в компьютерном зрении нужно исключительно с помощью встроенных векторных функций NumPy и OpenCV

- **Вопрос 1.** Кадр 1920×1080 (Full HD, также обозначается как 1080p), три канала, `uint8`. Сколько байт в памяти? А тридцать таких кадров в секунду?
- **Вопрос 2.** JPEG качества 95 «почти без потерь». Какая доля пикселей изменится после одного сохранения: около 1 %, 10 % или 40 %?

В формате JPEG параметр качества (quality) задаётся целым числом по шкале от 1 до 100.
100 — максимальное качество (минимальное сжатие, огромный вес файла).
95 — стандартный предел профессионального сжатия «почти без потерь» (visually lossless).

- **Ответ 1.** Сколько байт занимает один кадр? Мат. расчёт строится на перемножении всех пикселей, каналов и размера одного элемента (`uint8` = 1 байт): ширина $\times$ высота $\times$ каналы $\times$ 1 байт. $1920 \times 1080 \times 3 \times 1 = 6\,220\,800$ байт (≈ 6.2 МБ). Сколько занимают 30 FPS? ≈ 187 МБ/с!
- **Ответ 2.** Правильный ответ: около 40 % (а в некоторых типах изображений, вроде плавных градиентов или шумов — до 90–100 %). Это немного шокирует: «Казалось бы, если качество 95 % и глазом потерь не видно, значит, измениться должно около 1 % или максимум 10 % пикселей». Но математически это работает иначе:

Разница между визуальным и численным: человеческий глаз не заметит, если цвет пикселя изменился с красного (0, 0, 255) на (0, 1, 254). Для глаза это один и тот же цвет.

Матрицы в Computer Vision: для OpenCV/NumPy такое изменение — это другие числа. Если вычесть исходную матрицу из сохранённой в JPEG (`img_origin - img_jpeg`), вы увидите, что мат. значения поменялись у огромной части пикселей (около 40 % и более).

Последствия: если сохранить обучающую выборку в JPEG (даже с качеством 95), а затем начать обучать нейросеть или настраивать жёсткие фильтры порогов цветов (`cv2.inRange`), ваши алгоритмы могут начать сбоить или выдавать другие результаты. Поэтому промежуточные этапы обработки картинок в CV всегда сохраняют в форматы сжатия без потерь (lossless) — например, в PNG или TIFF.

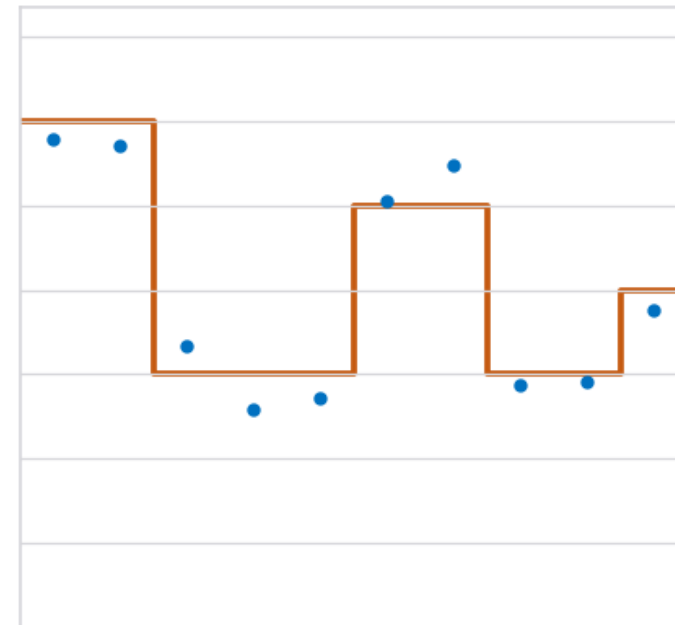
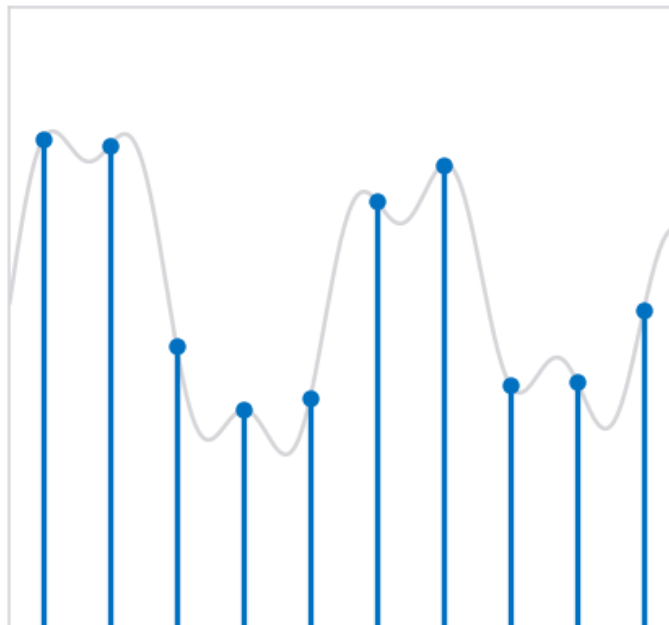
2

Изображение и видео как данные

- Оцифровка, массив, тип, гамма, разрешение, срезы, ресемплинг, сжатие, видео

Оцифровка: сетка пикселей и биты

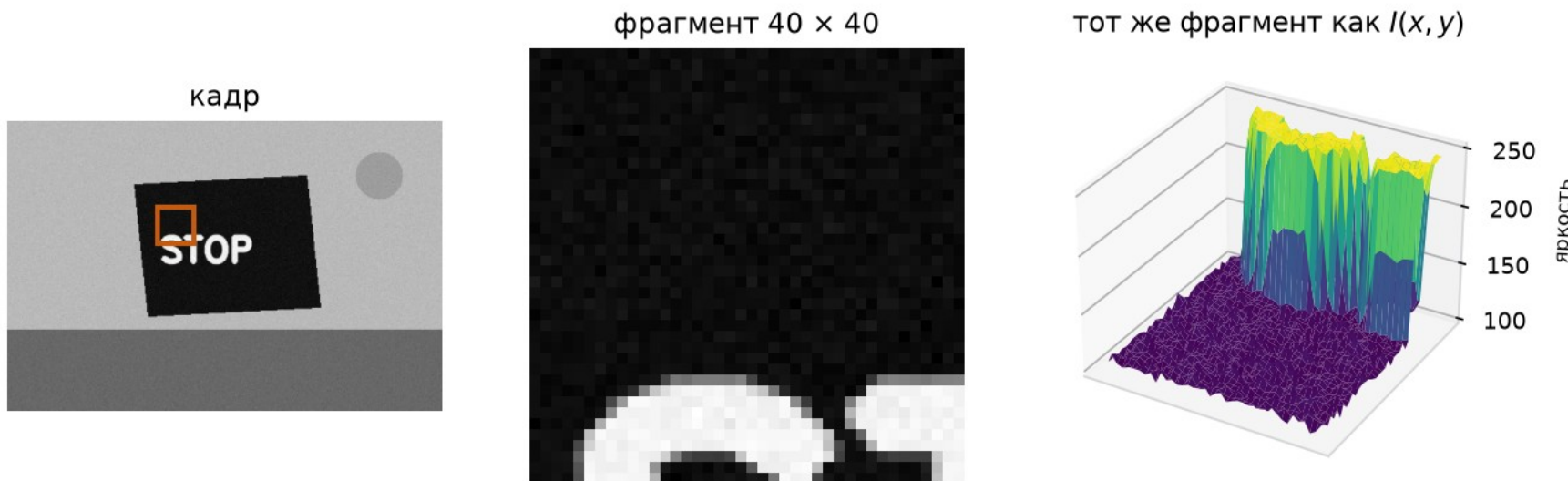
Оцифровка = дискретизация по координате (сетка пикселей) + квантование по яркости (биты)
непрерывный сигнал $I(x)$ дискретизация: только в узлах сетки квантование: 3 бита, 8 уровней



Оцифровка — две независимые потери: сетка (что между пикселями — неизвестно) и биты (что между уровнями — неизвестно). Первая задаёт разрешение, вторая — битность. Обе необратимы: никакой обработкой не восстановить то, что не попало в узлы сетки.

© собственная иллюстрация

Изображение — функция двух переменных



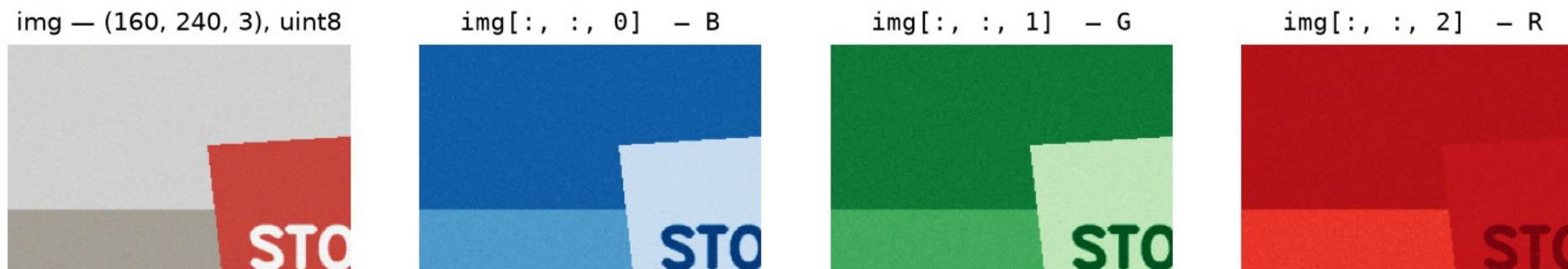
Тот же фрагмент как картинка и как поверхность $I(x, y)$

- Функция $I(x, y)$ — яркость в точке; для цвета таких функций три
- **Что это значит физически:** край объекта — место, где функция быстро меняется; её производная — тема лекции L7
- Непрерывная модель — для математики, дискретный массив — для кода; курс живёт в обеих парадигмах

© собственная иллюстрация

...и трёхмерный массив $H \times W \times C$

Первая ось — строки (H), вторая — столбцы (W), третья — каналы. Точка (x, y) — это `img[y, x]`



Кадр и три его канала по отдельности

- Первая ось — **строки** (H), вторая — **столбцы** (W), третья — каналы цвета
- Точка (x, y) — это `img[y, x]`: порядок обратный, и это частая ошибка курса наряду с BGR
- Начало координат — **левый верхний** угол, ось y направлена вниз

Три оси дают сразу три характеристики: форму, порядок осей и каналы. Четвёртая характеристика — чем закодировано одно значение — это `dtype`.

© собственная иллюстрация

dtype: чем закодирован один пиксель

dtype	Диапазон	Где встречается
uint8	0...255	обычные фото и видео — 99 % курса
uint16	0...65535	RAW с камеры, карты глубины, медицина
float32	любой	все вычисления внутри алгоритмов
bool	0 / 1	маски: «этот пиксель — объект»

Правило курса: храним в `uint8`, считаем во `float32`, маски — `boolean`

Битность — сколько уровней различимо

Битность — сколько различных уровней яркости, а не «качество»

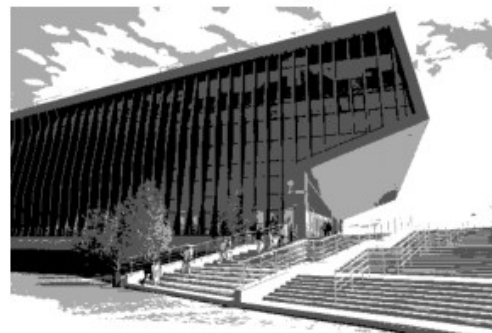
8 бит — 256 уровн.



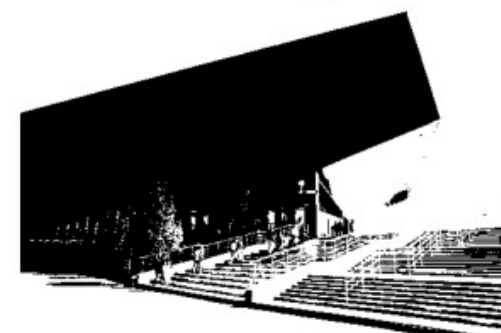
4 бит — 16 уровн.



2 бит — 4 уровн.



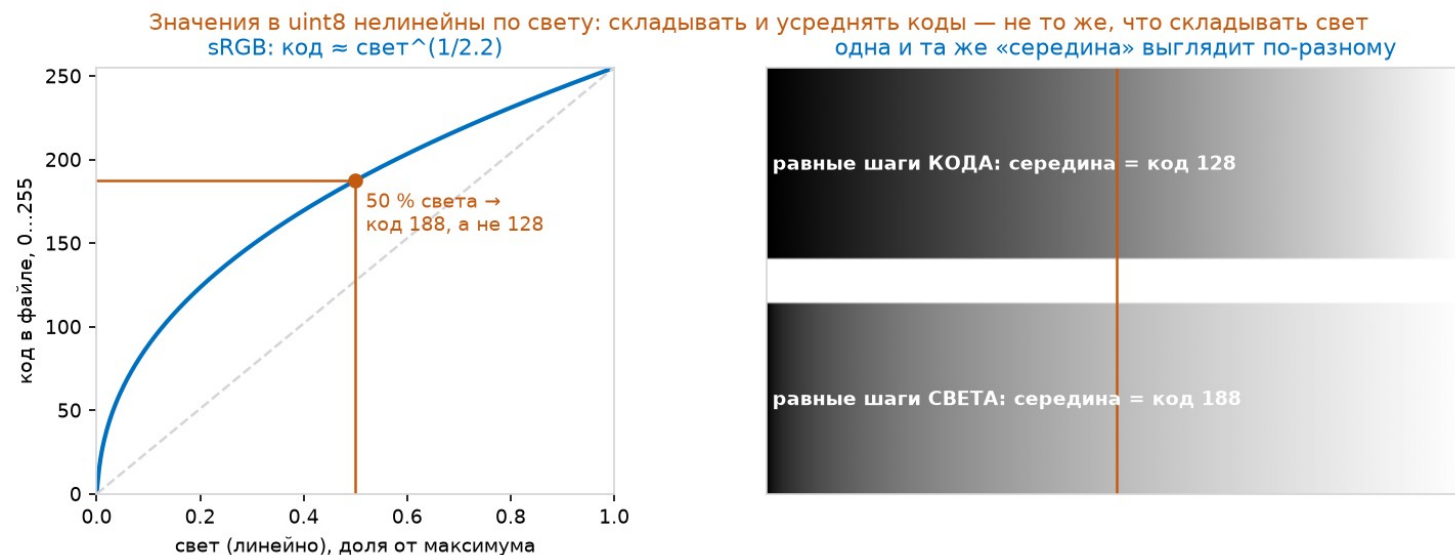
1 бит — 2 уровн.



Одна и та же сцена при 8, 4, 2 и 1 бите на пиксель. Битность — число различных уровней яркости, а не «качество». Восемь бит — 256 уровней, вроде хватает; но загляните в тени или растяните контраст — и появятся ступеньки: 8 бит уже мало. RAW камеры — 12–14 бит именно ради теней. Вывод: биты — это запас уровней. Но тратится этот запас нелинейно — и здесь на сцену выходит гамма.

© фото — здание Университета Иннополис; собственная иллюстрация

Код 128 — не половина света

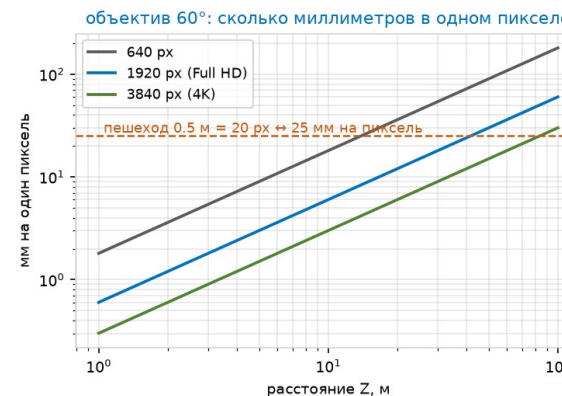
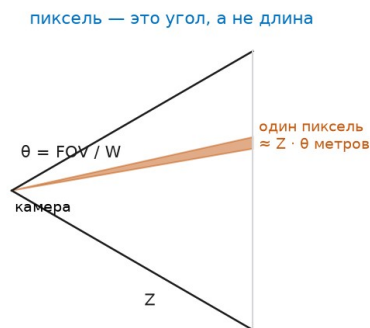


sRGB нелинеен: код 128 — это не половина света, а лишь 22 % света. Половина света — код 188. sRGB нелинеен: он упаковывает коды плотнее в тенях, где глаз чувствительнее. Следствие: складывать и усреднять коды uint8 — значит складывать не свет; отсюда неверные градиенты, средние и пороги в чужом коде. Подробности — в лекции L3

sRGB — это стандартное цветовое пространство, созданное компаниями HP и Microsoft для использования на мониторах, принтерах и в интернете.

© собственная иллюстрация

Пиксель — это угол, а не длина



Один пиксель — угол FOV / W ; сколько это метров, зависит от расстояния

- Объектив 60°, 1920 px — **0.6 мрад** на пиксель: 6 мм на 10 м, 6 см на 100 м
- Детектору нужно ≥ 20 px на объект: пешеход 0.5 м виден Full HD до **42 м**, 4K — до **83 м**
- Больше пикселей — дальше видно, но вчетверо больше данных и меньше света на пиксель

У пикселя нет длины, у него есть угол: FOV , делённый на число столбцов. Объектив 60° на 1920 колонок — 0.6 мрад на пиксель: на 10 м это 6 мм, на 100 м — 6 см. Детектору нужно ≥ 20 пикселей на объект: пешеход шириной полметра виден Full HD до 42 м, 4K — до 83 м. То есть «разрешение» — не свойство камеры, а свойство задачи: дистанция, размер объекта, объектив. И тут разрешение оказывается не тем же, что детализация.

© собственная иллюстрация

Разрешение — не то же, что детализация

- Деталь должна занимать ≥ 2 пикселя (теорема отсчётов): мельче — станет шумом и муаром
- 4K с плохой оптикой хуже HD с хорошей: **резкость задаёт объектив**, матрица лишь её пересчитывает
- Кадр 4K — **25 МБ** uint8; тридцать таких в секунду — 746 МБ/с: разрешение оплачивается временем

Важен размер объекта в пикселях, а не размер кадра

Разрешение оплачивается данными: кадр 4K — 25 МБ, тридцать в секунду — 746 МБ/с. Важен размер объекта в пикселях, а не размер кадра. К цене данных вернёмся в видео-бюджете; а сначала — каналы и их порядок.

Каналы и порядок: ловушка BGR



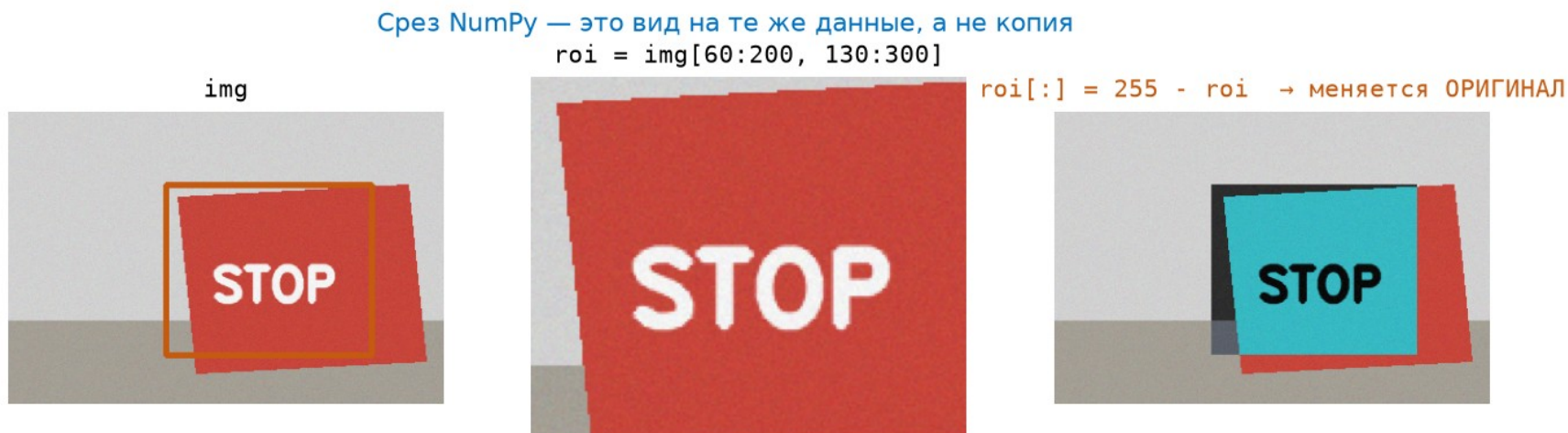
Один и тот же массив, показанный как BGR и как RGB

- `cv2.imread` возвращает **BGR** — так исторически сложилось из-за формата BMP
- `matplotlib`, PIL, браузеры, все веб-форматы — **RGB**
- Лечится одной строкой `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`; в курсе это делает `viz.show`

Один массив, показанный как BGR и как RGB: BGR делает небо жёлтым. `cv2.imread` возвращает BGR — наследие BMP; `matplotlib`, PIL, браузеры ждут RGB. Лечится одной строкой `cvtColor`, в курсе это делает `viz.show`. Ошибка очень частая. Следующая ловушка — срезы.

© собственная иллюстрация

Срезы и ROI: вид, а не копия



ROI как срез; запись в срез меняет оригинал

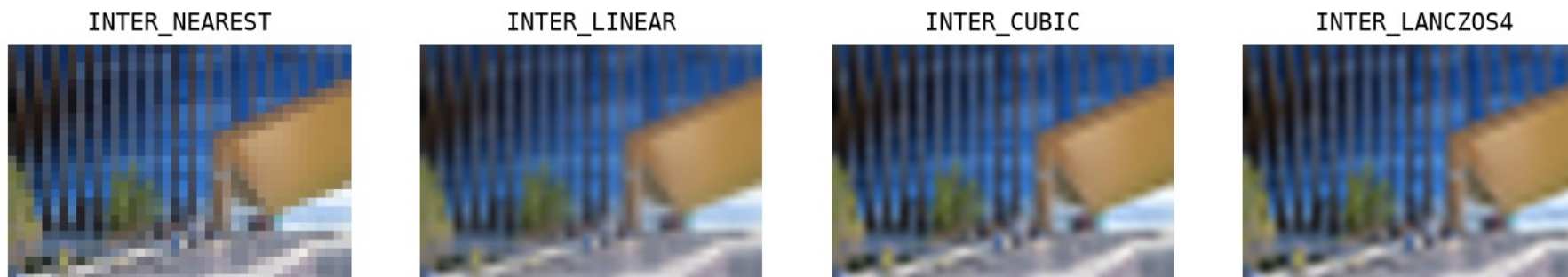
- `roi = img[y0:y1, x0:x1]` — **вид** (view) на ту же память, копирования нет
- Запись в срез меняет исходный массив; нужна независимая копия — `img[...].copy()`
- Именно поэтому срез бесплатен по времени: NumPy не копирует 7 МБ ради окна 200×300

Срез `roi = img[y0:y1, x0:x1]` — вид на ту же память, копирования нет. Запись в срез меняет исходный массив — убедимся в демо 2.1. Именно поэтому срез бесплатен: NumPy не копирует 7 МБ ради окна 200×300 . Нужна независимая копия — `.copy()`.

© собственная иллюстрация

Размер и флаги интерполяции

Один и тот же кадр 40×28 , увеличенный до 320×220 разными флагами



Кадр 40×28 , увеличенный четырьмя разными флагами

- Увеличение и уменьшение — **разные задачи**: при увеличении придумываем значения между узлами, при уменьшении — выбрасываем
- Флаг по умолчанию `INTER_LINEAR` подходит не всегда; выбор проверяется числом — следующий слайд

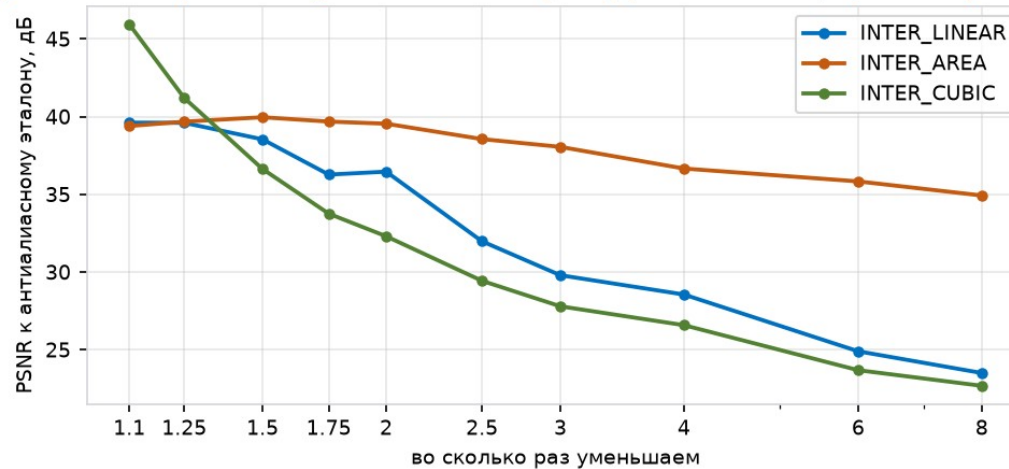
Кадр 40×28 , увеличенный четырьмя флагами: `nearest` — ступеньки, `linear` — мыло, `cubic` — резче, но с ореолами.

Увеличение и уменьшение — разные задачи: при увеличении придумываем значения между узлами, при уменьшении — выбрасываем отсчёты, и без усреднения получаем алиасинг. Флаг по умолчанию `INTER_LINEAR` подходит не всегда.

© фото — здание Университета Иннополис; собственная иллюстрация

Уменьшение: измерение вместо правила

«Для уменьшения берите INTER_AREA» — заметно (≥ 1 дБ) лишь с $\approx 1.5\times$; при $1.1\times$ лучше INTER_CUBIC



Измерено на этом кадре против независимого эталона (Lanczos из Pillow): при $1.1\times$ выигрывает INTER_CUBIC (46 дБ), INTER_AREA впереди на ≥ 1 дБ только с $1.5\times$, при $4\times$ — на 9 дБ. Правило верно, но не всегда

Есть среди разработчиков миф (или слепое правило) о том, что для любого уменьшения изображений в OpenCV нужно использовать интерполяцию INTER_AREA. Измерили против независимого эталона — Lanczos из Pillow. При $1.1\times$ выигрывает INTER_CUBIC — 46 дБ; INTER_AREA сильнее на ≥ 1 дБ только с $1.5\times$, и при сильном уменьшении при $4\times$ — выигрывает на 9 дБ. Если вы сжимаете большое изображение в маленький эскиз (например, из 4K делаете превью) — используйте `cv2.INTER_AREA`.

© собственная иллюстрация

Флаг	Для чего	Почему
INTER_NEAREST	маски и метки классов	не изобретает новых значений: класс 2.5 не появится
INTER_AREA	уменьшение от $\approx 1.5\times$	усредняет площадь — нет алиасинга; при $1.1\times$ не нужен
INTER_LINEAR	увеличение, значение по умолчанию	быстро и приемлемо; на большом увеличении мыло
INTER_CUBIC, LANCZOS4	увеличение и лёгкое уменьшение	резче, но дают ореолы у контрастных границ

Какой флаг когда. Рабочая таблица. NEAREST — маски и метки классов: не изобретает значений, класса 2.5 не появится. AREA — уменьшение от $\approx 1.5\times$: усредняет площадь, нет алиасинга. LINEAR — увеличение по умолчанию: быстро и приемлемо. CUBIC и LANCZOS4 — увеличение и лёгкое уменьшение: резче, но ореолы у контрастных границ.

JPEG теряет информацию необратимо: артефакты 8×8 — это то, что дальше примут за края

quality=95 · 124 КБ



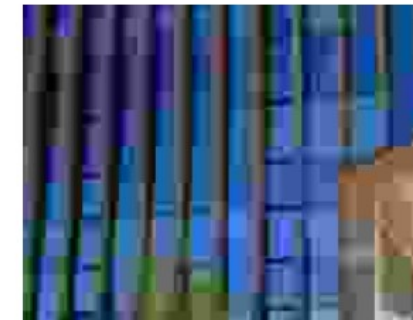
quality=50 · 37 КБ



quality=20 · 21 КБ



quality=5 · 9 КБ



Один фрагмент при разном качестве JPEG

- JPEG теряет **необратимо**: блоки 8×8 , косинусное преобразование, квантование. При качестве $q = 95$ меняются **44 %** пикселей
- Артефакты блоков — это края, которых нет в сцене: их найдёт Canny в L7, а вычитание фона в лекции L13 примет за движение

Поэтому промежуточные результаты — только PNG. Но у JPEG есть ещё один сюрприз — про цвет.

© фото — здание Университета Иннополис; собственная иллюстрация

JPEG хранит цвет вполовину

JPEG хранит цвет в половинном разрешении (4:2:0): деталь, живущая только в цвете, размывается

оригинал (PNG)



его яркость Y: текста почти нет



JPEG quality=95



JPEG quality=75



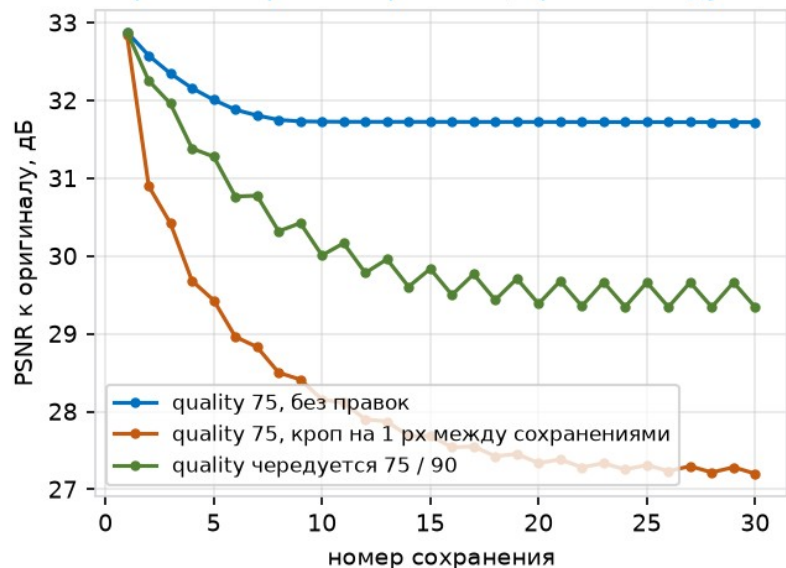
Яркость JPEG хранит в полном разрешении, цвет — в половинном (4:2:0). Деталь, отличающаяся только цветом, размывается на 2 px и теряет тонкие линии. Для цветовой сегментации (L3) это источник ложных границ

Вывод для практики: если задача зависит от тонких цветовых линий — не доверяйте JPEG, снимайте PNG или RAW.

© собственная иллюстрация

Поколения JPEG: где копятся потери

главная потеря — в первом сохранении; правки между ними её копят



1-е сохранение, $q = 75$



30-е, с кропом между



При том же quality и той же сетке 8×8 пересохранение почти идемпотентно: $32.9 \rightarrow 31.7$ дБ и стоп. Потери копятся, когда между сохранениями кадр кропают или меняют quality — то есть при любом реальном редактировании

Интуиция говорит: «каждое пересохранение ухудшает». Потери копятся, когда между сохранениями кадр кропают, сдвигают или меняют quality — то есть при любом реальном редактировании.

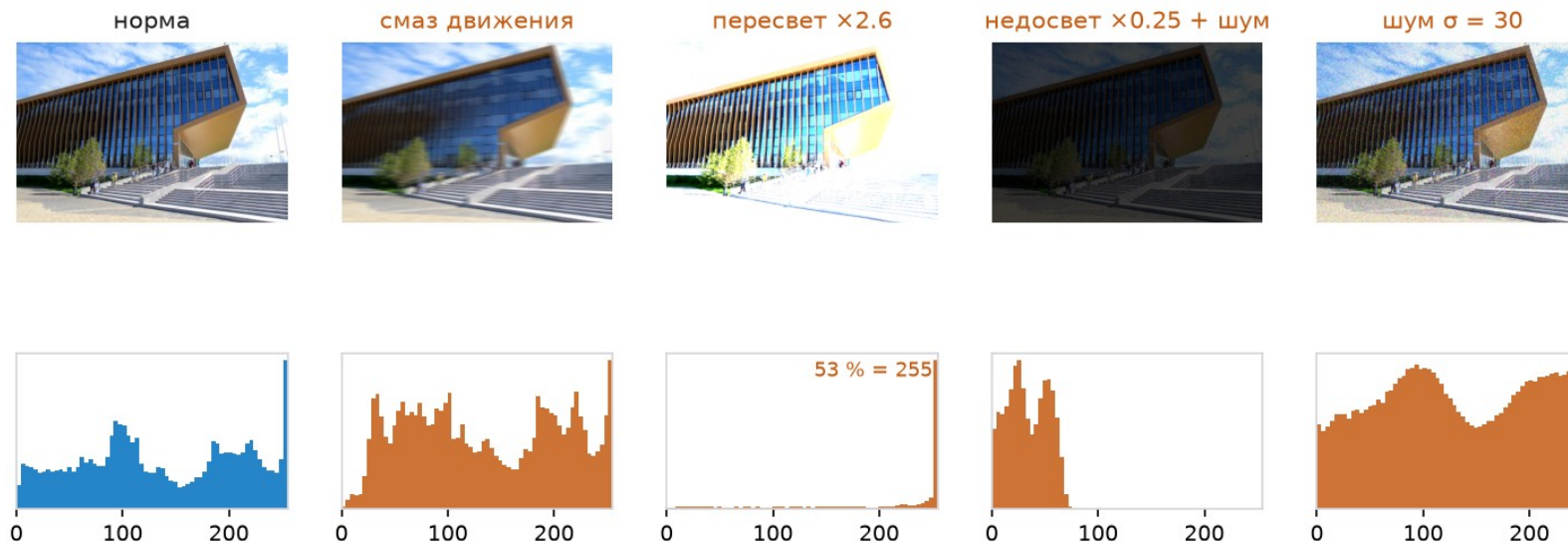
© фото — здание Университета Иннополис; собственная иллюстрация

Формат	Сжатие	Когда брать
PNG	без потерь	маски, эталоны, промежуточные и итоговые результаты
JPEG	с потерями, цвет вполонину	исходные фото, которые вы не контролируете
TIFF	без потерь, 16 бит и больше	RAW, глубина, научные и медицинские съёмки
MP4 / H.264	с потерями, между кадрами	видео; кадр уже испорчен кодеком до вашего кода

Рабочая таблица. PNG — маски, эталоны, любые промежуточные результаты: без потерь. JPEG — только исходные фото, которые вы не контролируете. TIFF — RAW, глубина, 16 бит: наука и медицина. MP4/H.264 — видео; но помните: кадр уже испорчен кодеком до вашего кода. Выбор формата — инженерное решение с ценой в данных.

Что необратимо: смаз и пересвет

Что лечится обработкой: шум и недосвет — частично. Что не лечится никогда: смаз и пересвет (значения упёрлись в 255)

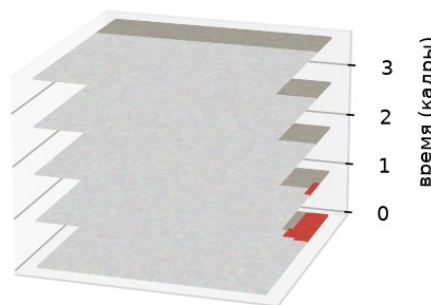


Обработка вернёт часть шума и недосвета; смаз и пересвет не вернёт никто: у пересвеченного кадра 53 % пикселей упёрлись в 255, информация уничтожена на сенсоре: обработка не восстановит. Отсюда L2

Шум и недосвет частично вытягиваются; насыщение — нет. Отсюда иерархия ответственности: ранние звенья пайплайна важнее любых алгоритмов.

© фото — здание Университета Иннополис; собственная иллюстрация

Видео — не «набор картинок», а массив $T \times H \times W \times C$:
по третьей оси лежит движение



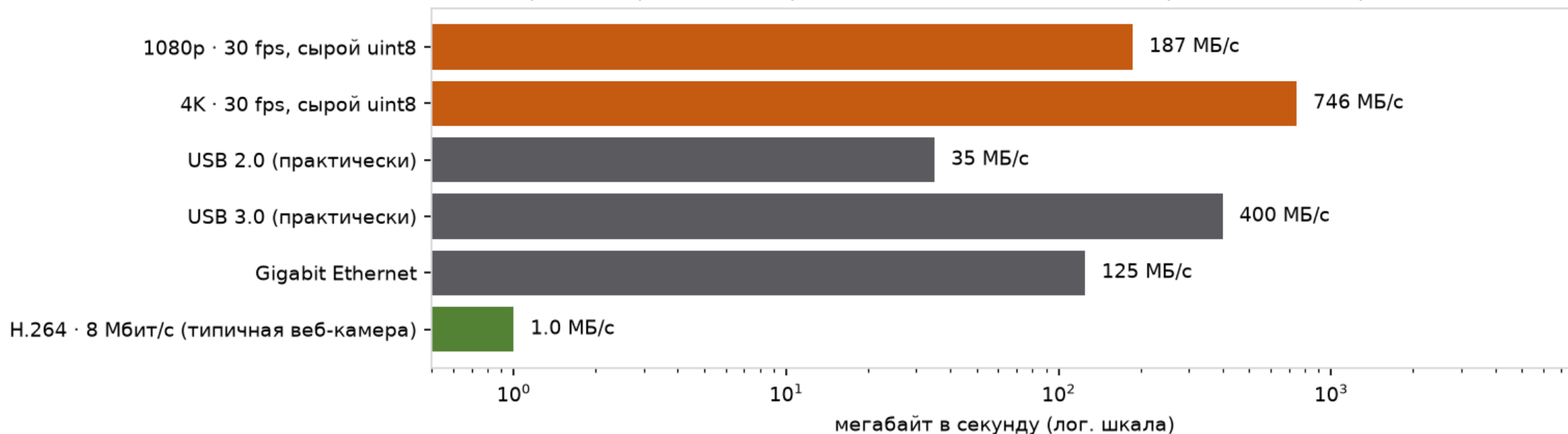
Видео как объём $T \times H \times W \times C$

- Видео — не «папка с картинками», а массив с **третьей осью — временем**
- Именно вдоль неё в блоке V мы будем искать движение: оптический поток — производная по t
- Кадры **не независимы**: кодек хранит разности, и соседние кадры уже связаны его предсказанием

Важный нюанс: кадры в файле не независимы — кодек хранит разности и предсказания, поэтому видео — не «папка с картинками», а связанный поток.

© собственная иллюстрация

Сырое 1080p-видео не проходит даже по USB 2.0: камера сжимает кадр ещё до вас



Сырое 1080p — 187 МБ/с, 4K — 746 МБ/с; USB 2.0 даёт 35. Камера сжимает кадр ещё до вашего кода (JPEG, H.264), и вы получаете уже испорченные пиксели: блочные артефакты, урезанный цвет.

Поэтому вопрос «каким кодеком снято» не менее важен, чем «каким объективом».

© собственная иллюстрация

- `cv2.VideoCapture` — и файл, и камера, интерфейс один; индекс 0 — встроенная камера
- Заголовок **врёт или молчит**: `CAP_PROP_FPS` и `FRAME_COUNT` для камер и потоков недостоверны — измеряйте
- Захват надо **освободить**, иначе камера занята до перезапуска ядра
- Переход к кадру N в H.264 — декодирование от ближайшего опорного кадра: медленно и неточно

В курсе `cvcourse.io.video_frames` освобождает захват сам

- Изображение — функция для математики и массив для кода; оцифровка — сетка плюс биты, и обе теряют
- Пять характеристик массива: форма, тип, диапазон, каналы, порядок осей — и гамма поверх всего
- Формат, флаг и кодек — решения, которые проверяются числом, а не репутацией

Дальше — четыре измерения кодом: память, ресемплинг, сжатие, видео

Демо 2.1 — срез это вид на память

python

```
1 roi = img[300:500, 600:900]          # y: 300...499, x: 600...899
2 print(roi.shape, roi.base is img)    # вид, а не копия
3
4 roi[:] = 255 - roi                    # инвертировали ТОЛЬКО срез...
5 print("...но img[300, 600] стал", img[300, 600])
6
7 patch = img[300:500, 600:900].copy() # копия живёт отдельно
8 patch[:] = 0
9 print("после patch[:] = 0 в img:", img[300, 600])
```

roi.shape (200, 300, 3) · roi.base is img: True ·
img[300, 600] был [28 52 80], стал [227 203
175] · после patch[:] = 0 остался [227 203 175]

Вызов

`img[y0:y1, x0:x1]`

`roi.base`

`.copy()`

Что делает

срез: новый объект-массив на **той же памяти**; строки до y1, столбцы до x1 не включая

исходный массив, чью память разделяет вид; у копии — None

выделяет новую память и переписывает данные; связь с оригиналом разорвана

Демо 2.2 — флаг `resize` в числах

python

```
1 from PIL import Image
2 from cvcourse import metrics
3 pil = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
4 for k in (1.1, 4.0):
5     size = (round(img.shape[1] / k), round(img.shape[0] / k))
6     ref = np.asarray(pil.resize(size, Image.LANCZOS)) # чужой эталон
7     ref = cv2.cvtColor(ref, cv2.COLOR_RGB2BGR)
8     for name in ("INTER_LINEAR", "INTER_AREA", "INTER_CUBIC"):
9         f = getattr(cv2, name)
10        got = cv2.resize(img, size, interpolation=f)
11        print(k, name, "%.1f дБ" % metrics.psnr(ref, got))
```

1.1×: LINEAR 39.2 · AREA 38.9 · CUBIC 46.4 дБ
— 4×: LINEAR 27.9 · AREA 37.2 · CUBIC 25.9 дБ

Вызов

`cv2.resize(img, (W, H), interpolation=...)`

Что делает

пересчитывает кадр на новый размер; размер — **ширина, высота**, обратно `shape`

`metrics.psnr(a, b)`

сходство в децибелах: > 40 — разницы не видно, < 25 — артефакты заметны глазом

Демо 2.3 — сжатие в числах

```
1 print("сырой массив: %d КБ" % (img.nbytes // 1024))
2 ok, png = cv2.imencode(".png", img)          # в память, не на диск
3 print("PNG: %d КБ, без потерь" % (len(png) // 1024))
4 for q in (95, 75, 50, 20):
5     prm = [cv2.IMWRITE_JPEG_QUALITY, q]
6     ok, buf = cv2.imencode(".jpg", img, prm)
7     back = cv2.imdecode(buf, cv2.IMREAD_COLOR)
8     changed = (back != img).any(axis=2).mean() * 100
9     kb, ps = len(buf) // 1024, metrics.psnr(img, back)
10    print("q=%d: %d КБ, PSNR %.1f дБ, изменено %.0f %"
11          % (q, kb, ps, changed))
```

python

сырой 7194 КБ · PNG 3759 КБ · q=95: 694 КБ,
49.5 дБ, изменено 44 % · q=75: 385 КБ, 39.8
дБ, 87 % · q=20: 141 КБ, 30.2 дБ, 100 %

Вызов

`cv2.imencode(ext, img, params)`

`cv2.imdecode(buf, флаг)`

`(back != img).any(axis=2)`

Что делает

кодирует массив в байты файла прямо в памяти; `params` — пары флаг–значение

декодирует байты обратно в массив — то же, что прочитав файл с диска

маска пикселей, у которых изменился **хотя бы один** канал

Демо 2.4 — заголовок против измерения

python

```
1 import os, time
2 from cvcourse import io as cio
3 path = "data/l01/synth-1080p.mp4"          # 90 кадров, 1080p
4 print(cio.video_info(path))                # что написано в заголовке
5 t0, n = time.perf_counter(), 0
6 for frame in cio.video_frames(path):        # захват освободится сам
7     n += 1                                  # здесь была бы обработка
8 dt = time.perf_counter() - t0
9 raw_mb = n * frame.size / 2**20            # H · W · 3 байт на кадр
10 print("%d кадров за %.2f с → %.0f к/с" % (n, dt, n / dt))
11 mb = os.path.getsize(path) / 2**20
12 print("сырых %.0f МБ, файл %.1f МБ" % (raw_mb, mb))
```

заголовок: 90 кадров, 30 fps, 1920 × 1080, 3.0
с · измерено: 90 кадров за 0.45 с → 199 к/с
только декодирование · сырых 534 МБ, файл
2.2 МБ — сжатие в 246 раз

Вызов

Что делает

`cio.video_info(path)`

читает заголовок: кадры, fps, размер — **не измеряет**, а пересказывает

`cio.video_frames(path)`

генератор кадров поверх `cv2.VideoCapture`; `release()` вызывается сам, даже при `break`

`frame.shape`

у кадра та же форма (H, W, 3), что у картинки: видео — это те же массивы

Фрагмент

- `img = cv2.imread("фото.jpg")`
- `plt.imshow(img)`
- `img[0:50, 0:50] *= 2`
- `small = cv2.resize(img, img.shape[:2])`

Четыре ошибки

- `imread` не проверен на `None`, а кириллица в пути его гарантирует
- BGR показан как RGB
- `*= 2` переполняет `uint8`: яркое станет тёмным
- `resize` ждёт (W, H), а `shape` даёт (H, W)

Ошибки первых двух недель (1/2)

ПОДВОДНЫЕ КАМНИ

- **BGR против RGB:** `imread` даёт BGR, `plt.imshow` ждёт RGB; в курсе — `viz.show`
- ``uint8`` переполняется молча: $200 + 100 = 44$; считать во `float32`, возвращать через `clip`
- ``shape`` — это ``(H, W, C)``, точка — `img[y, x]`, а `resize` и `VideoWriter` ждут `(W, H)`

Ошибки первых двух недель (2/2)

ПОДВОДНЫЕ КАМНИ

- **Срез — не копия:** правка `roi` меняет оригинал; нужна копия — `.copy()`
- `cv2.imread` возвращает `None` вместо ошибки: кириллица в пути, опечатка, не скачан файл
- Маски ресемплим только `INTER_NEAREST`; цикл по пикселям на Python — в 150–700 раз медленнее

KEY TAKEAWAYS

- Зрение — **обратная задача**: снимок уже потерял глубину, спектр и время; вернуть их — цель курса
- Алгоритм получает **только числа**; всё, что «видно», надо вычислить, и попиксельно это не делается
- Изображение — функция $I(x, y)$ для формул и массив $H \times W \times C$ для кода; в `uint8` спрятана гамма
- Ошибка раннего звена **необратима**: пересвет, смаз, JPEG — никакая обработка их не вернёт
- Флаг, формат, кодек и «заявленный FPS» проверяются **измерением**, и сегодня мы это сделали

САМОПРОВЕРКА

- Кадр 4K `uint8`, три канала — сколько байт, и почему ответ «8 мегапикселей» неверен?
- Почему `INTER_LINEAR` нельзя применять к маске сегментации, а `INTER_AREA` не помогает при `1.1x`?
- Что вернёт `img[100:200, 50:60].shape`, и изменится ли `img`, если этому срезу присвоить 0?
- Два разных спектра дают одинаковые R, G, B — как это называется и почему камера этого «не замечает»?

Разбор — в начале L2, на слайде «Что было в прошлый раз»

Занятие 1 и домашнее задание 1

ДАЛЬШЕ

- **Занятие 1 «Изображение как массив»:** `describe`, нарезка патчей с `.copy()`, контактный лист, таблица флагов `resize` на своём фото
- **Мост к ДЗ 1:** нарезка, ресемплинг и контактный лист уходят в «Фотолабораторию» как есть
- **ДЗ 1 «Фотолаборатория»** — выдача W3, дедлайн W6

Окружение ставится **до занятия**: `docs/setup-guide.md`; проверка — `python tools/check_env.py`

REFERENCES

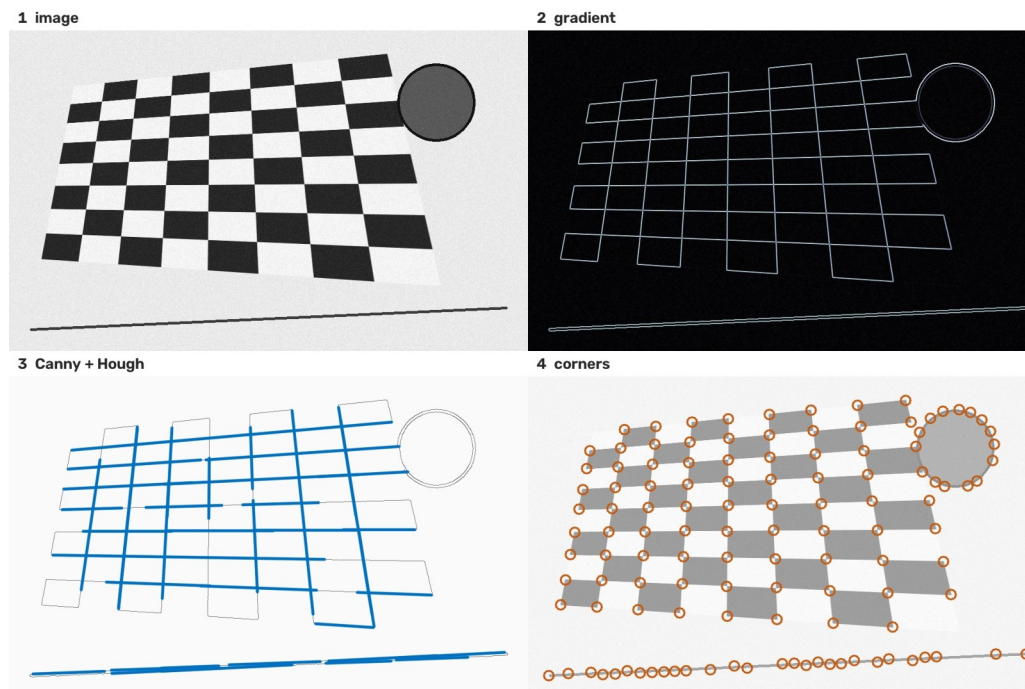
- Szeliski R. *Computer Vision: Algorithms and Applications*, 2nd ed., 2022 — гл. 1, 2.2–2.3: формирование изображения, сэмплинг, цвет. PDF свободен на szeliski.org
- Bradski G., Kaehler A. *Learning OpenCV 4*, 2019 — гл. 2–3: первые программы, Mat и массивы
- Poynton C. *Digital Video and HD*, 2nd ed., 2012 — гамма и субдискретизация цвета, без мифов
- Maimone M., Cheng Y., Matthies L. *Two years of Visual Odometry on the Mars Exploration Rovers*, J. Field Robotics, 2007
- Siegwart R., Nourbakhsh I. R., Scaramuzza D. *Introduction to Autonomous Mobile Robots*, 2nd ed., MIT Press, 2011 — гл. 4 «Perception»: зрение как датчик мобильного робота
- OpenCV docs, ветка **4.x**: `imread`, `resize`, `VideoCapture` · NumPy: *Indexing on ndarrays* — про виды и копии

К следующей паре обязательно: Szeliski, гл. 1 — двадцать пять страниц

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
пиксель (pixel)	отсчёт функции яркости в узле сетки: число 0...255 на канал; у него есть угол, но нет длины
разрешение (resolution)	число отсчётов по сторонам кадра, $W \times H$; не то же, что резкость — её задаёт объектив
битность (bit depth)	число различных уровней яркости: 8 бит — 256 уровней, RAW камеры — 12–14 бит
канал (channel)	одна из компонент цвета в массиве; в OpenCV порядок B, G, R — наследие формата BMP
гамма (gamma)	нелинейное кодирование света в код: половина света — код 188, а не 128
область интереса (ROI)	прямоугольный фрагмент кадра; в NumPy — срез, то есть вид на ту же память
вид (view)	массив, разделяющий память с другим; правка вида меняет оригинал, <code>.copy()</code> рвёт связь
ресемплинг (resampling)	пересчёт кадра на новую сетку; при уменьшении нужен антиалиасинг — <code>INTER_AREA</code>
интерполяция (interpolation)	оценка значения между узлами: ближайший сосед, билинейная, бикубическая, Lanczos
алиасинг (aliasing)	ложный узор при выборке реже двух отсчётов на деталь — муар на мелкой сетке
сжатие с потерями (lossy compression)	кодирование, необратимо отбрасывающее информацию: JPEG, H.264; PNG — без потерь
субдискретизация цвета (chroma subsampling)	цвет хранится в половинном разрешении (4:2:0) при полном разрешении яркости
метамерия (metamerism)	разные спектры дают одинаковые R, G, B: три числа не восстанавливают спектр
частота кадров (FPS)	кадров в секунду; заявленное в заголовке и измеренное в цикле — разные величины

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Окружение поставить **до занятия 1**: `docs/setup-guide.md`

Проверка одной командой: `python tools/check_env.py`

Демо-код лекции: `lectures/L01-intro/demo/`